Project #4 CS 3410– Spring 2026 <Emmett Bicknell & Brandon Aikman>

Requirements

Restate the problem specification, and any detailed requirements

We were tasked with solving the Rush Hour problem utilizing a BFS approach. We had to print out the minimum number of moves as well as the moves taken to reach the solution. Our game board was a 6 x 6 matrix with the exit being the right edge of the third row. For this puzzle, there were two vehicle types, cars (2 long) and trucks (3 long), each 1 wide. Vehicles could be oriented horizontallly or vertically, and would remain as such for the remainder of the puzzle.

Design

How did you attack the problem? What choices did you make in your design, and why? Show class diagrams for more complex designs.

As instructed, we used a breadth-first search approach. We had three key classes: Car, Node, and bfs. bfs handled our puzzle logic and solution implementation. Car allowed us to cleanly store information about each car, such as its length, color, orientation, and more. Finally, Node allowed us to store different BFS states, consisting of a key which represented its current state, a parent Node, its height, and its move. We utilized a 7 x 7 matrix to make indexing easier. Additionally, we had two helper functions to convert the state array into a String and vice versa, which proved helpful for initializing Nodes.

Security Analysis

State the potential security vulnerabilities of your design. How could these vulnerabilities be exploited by an adversary? What would be the impact if the vulnerability is exploited?

There are no known security vulnerabilities. The program cannot run any terminal commands so nothing bad should be able to happen.

Implementation

Outline any interesting implementation details.

We used a Queue (Java LinkedList implementation) for holding the states to check. We also used a HashMap to keep track of found states, which allowed us to cleanly check if a state had been visited or not. Finally, we used a Stack to push all moves of the solution into in reverse order, then popped them off to reveal the order of moves from start to end. This project highlighted the importance of using different data structures to solve an algorithm and also showed that understanding what data structure is best applied to a certain type of problem is crucial for solving problems efficiently.

Testing

Explain how you tested your program, enumerating the tests if possible. Explain why your test set was sufficient to believe that the software is working properly, i.e., what were the range of possibilities of errors that you were testing for.

There were about four main types of tests we used. Two common tests were 1 car red h 3 5, which simply tests if it can correctly identify a solved state, and 1 car red h 3 2, which checks to see if it can correctly move the red car to the end. We also used tests like 2 car red h 3 1 truck blue v 6 5, which tests to see if the algorithm can move a vehicle out of the way and then move the red car to the finish.

We also used the provided example problem, 8 car red h 3 2 car lime h 1 1 truck purple v 2 1 car orange v 5 1 truck blue v 2 4 truck yellow v 1 6 car lightblue h 5 5 truck aqua h 6 3.

Finally, once our program was showing promise, we used the Gradel test cases.

AI Use

How did you use generative AI in this project? Be specific!

We did not use generative AI on this project.

Summary/Conclusion

Present your results. Did it work properly? Are there any limitations? If it is an analysis-type project, this section may be significantly longer than for a simple implementation-type project.

Our project works properly. Gradel said we passed 5 out of 7 tests. However, our output for the two tests that were marked as failing was exactly the same as the desired output. We check it character-by-character and they were identical. We talked to Dr. G about this, and he said that the likely reason for this is that Gradel timed out when running these test cases. One limitation to our solution is that if posssible runtime is very limited, our code may exceed desired runtime for certain input cases.

Lessons Learned

List any lessons learned, especially in regards to AI use.

This project reinforced the lesson that it is very important to carefully plan out your approach before you start writing lines of code. It also was a reminder that it can be very helpful to make sketches to visualize what's going on, and to sanity-check the code.

Time Spent

Approximately how much time did you spend on this project?

Emmett spend about 15-20 hours on this project. Brandon spent 6.